Software Engineering Principles

CSI1007

Module – 4

Software Implementation

Module:4 Software Implementation

Structured coding Techniques-Coding Styles-Standards and Guidelines- Documentation Guidelines-Modern Programming Language Features: Type checking-User defined data types- Data Abstraction-Exception Handling- Concurrency Mechanism – Seven Steps of implementing software – Implementation Challenges and its resolution.

Structured Coding Techniques

- Structured programming is a technique in software engineering aimed at improving the clarity, quality, and development time of a computer program. It makes extensive use of **structured control flow constructs, block structures, and subroutines**.
- Sub-divide the **whole program into small modules** so that the program becomes easy to understand.
- The purpose of structured programming is to linearize control flow through a computer program so that the **execution sequence follows the sequence in which the code is written**.

Structured Coding Techniques

- The size of the **software increases as the lines of code multiply** during the coding process.
- Over time, **remembering the flow of the program** becomes nearly impossible.
- If one forgets the structure of the software and its underlying programs, files, and procedures, **sharing, debugging, and modifying the software becomes extremely difficult**.
- Structured programming is the solution to this problem.
- It encourages programmers to use subroutines and loops instead of simple jumps in the code, **leading to greater clarity and efficiency**.
- Structured programming also helps programmers **reduce coding time and organize their code properly.**

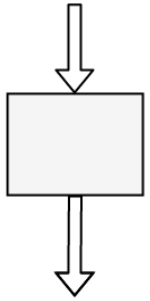
Structured Coding Techniques

- Why Needed ?
  - To **simplify the comprehension** of the program for the coder.
  - When a program is composed of thousands of statements and an issue arises, **pinpointing the error within the entire program can be challenging**
  - With structured programming, we have the advantage of being **able to swiftly identify the error and navigate directly to its location for rectification**.

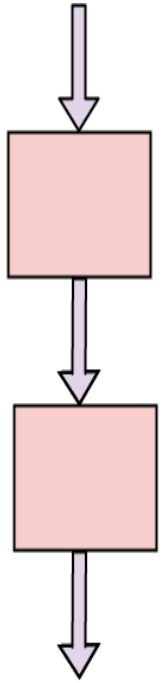
Structured Coding Techniques

- Code Block: A code block is structured with a **single-entry point** and **single exit point.** This enhances the readability, testability, and modifiability of the program.
- Sequence: A sequence of blocks is correct **if the exit conditions of each block match the entry conditions** of the following block.
- Alternation: If-then-else is frequently called alternation because there are alternative options. Each choice is a code block.
- Iteration: Iteration (while-loop) is organized as at right.
- Modularization: Breaking your code into functions makes it easier to read and maintain. **Each function has its purpose**, and you can focus on one at a time without getting overwhelmed

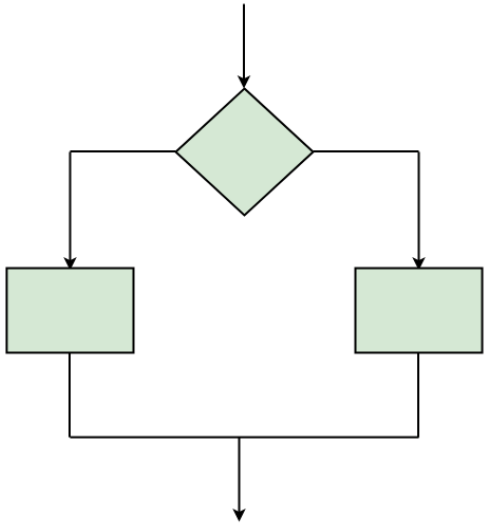
Structured Coding Techniques



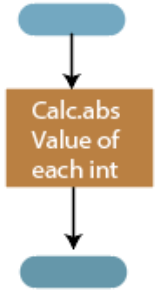
Rule1: Code block is structured



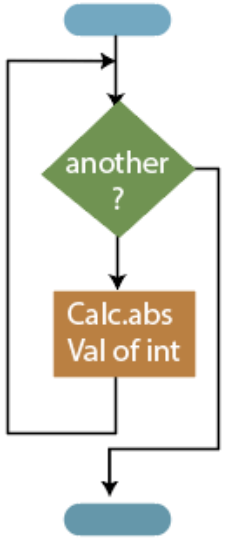
Rule2: A sequence of code blocks is structured



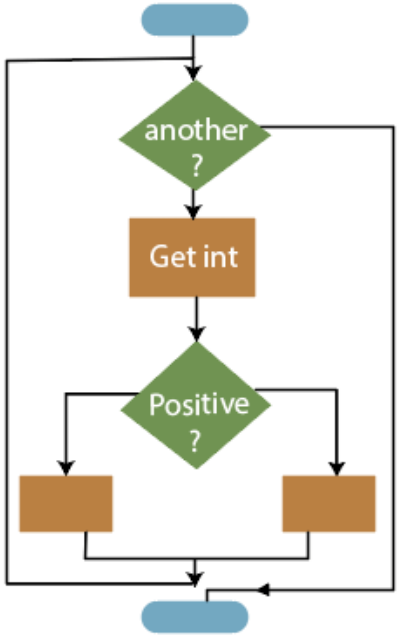
Rule 3: An alternation of code blocks is structured



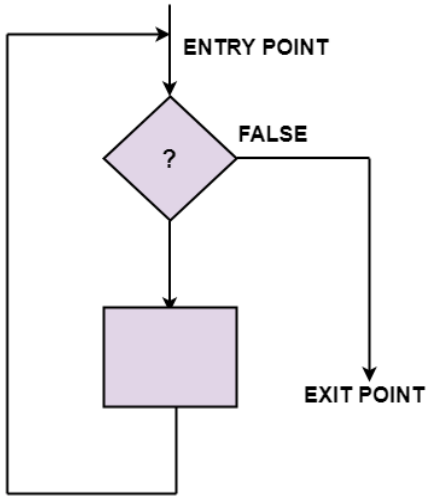
1. First View



2. Second View



3. Third View



Rule 4: Iteration of code blocks is structured

Structured Coding Techniques

- Major Concepts
  - **Top-down Analysis**:
  - Top-down analysis is a method of problem-solving where the **larger problem is broken down into smaller, manageable sub-problems.**
  - This approach is used **to understand and address the problem** at hand.
  - Each **sub-problem is solved individually**, and the solutions are then integrated to solve the overall problem.

Structured Coding Techniques

- Major Concepts
  - **Modular Programming:**
  - This is a design technique that involves **partitioning a program into separate sub-programs or modules**.
  - Each module is a **self-contained unit that performs a specific task** within the overall program.
  - Modular programming **discourages the use of 'goto' statements, which can make the program flow difficult** to trace. Instead, it promotes a structured approach where control flow is clearly defined and easy to follow.

Structured Coding Techniques

- Major Concepts
  - **Coding Structure/Control Structure:**
  - Structured coding is a **method of organizing code in a hierarchical manner,** where modules are subdivided into smaller units of code based on their execution order.
  - The control structure in structured programming is used to **dictate the flow of the program**.
  - In structured coding, the control structure is used to **organize instructions in a pattern** that is easy to understand and follow.

Structured Coding Techniques

- Major Concepts
  - **Stepwise refinement**
  - This is a method of developing a system where you start with a simple, general outline of the system and **gradually add more detail and complexity.**
  - For example, if you were writing a program to sort a list of numbers, you might start with a simple outline like “read in the numbers, sort the numbers, print out the sorted numbers”.
  - Then, you would gradually refine this, adding details about how to read in the numbers, what sorting algorithm to use, and how to print out the numbers

Structured Coding Techniques

- Major Concepts
  - **Data structures**
  - These are **ways of organizing and storing data** in a program.
  - For example, an **array** is a data structure that stores a fixed-size sequence of elements of the same type.
  - A **linked list** is a data structure that stores a sequence of elements, but allows for efficient insertion and removal of elements.
  - A **tree** is a data structure that stores elements in a hierarchical structure, with one root element and a set of subtrees.

Structured Coding Techniques

- Coding
  - **Procedural Programming**
  - This is a programming paradigm that **structures programs as sequences of instructions or procedures**.
  - Each procedure, similar to a function, can **manipulate global data and maintain local data** that is not accessible outside its scope.
  - Procedural programming languages, such as Fortran and ALGOL, encapsulate concepts that remain fundamental in contemporary programming languages.

Structured Coding Techniques

- Coding
  - **Procedural Programming – Principles**
  - Programs are **composed of sequences of instructions**, **with minimal abstraction** between the code and the machine.
  - Procedures, defined as logical blocks of instructions, **can be invoked from any part of the code**.
  - A procedure can accept arguments, return results to the caller, and access or modify global scope variables.
  - Procedural languages **employ block-based control flow**, avoiding 'goto' commands, and adhere to structured programming practices.

Structured Coding Techniques

- Coding
  - **Object Oriented Programming**
  - OOP is a programming paradigm that models a **program as a collection of objects or resources** that receive commands.
  - In an object-oriented language, a data resource is created and passed to processing commands.
  - For instance, where a procedural programmer might use “Print(object)”, an OOP programmer would use “Object.Print()”.

Structured Coding Techniques

- Coding
  - **Model-Based Programming:**
  - This programming approach is commonly **seen in database query languages.**
  - Code units in database **programming are associated with steps in database access and are updated** or executed when those steps occur.
  - The code's **structure is determined by the database** and its access structure.
  - Other examples of model-based structures include Reverse Polish Notation (RPN), which facilitates efficient resolution of complex expressions, and quantum computing, which requires a specific model to organize steps provided by the language

Coding Styles -Standards and Guidelines

- Coding standards, also known as coding guidelines or **programming style guides**, are **rules and conventions** that **developers follow** when writing code.
- These rules define the characteristics of the code necessary to **maintain a uniform codebase** and facilitate collaboration among developers.
- They ensure **consistency, readability, maintainability, and reliability** in software development.

Coding Styles-Standards and Guidelines

- Coding standards are important for several reasons.
  - They help **prevent bugs and security holes** due to bad code.
  - They **make it simpler for new people** to understand the codebase.
  - If there's a standard format, it's not as tough to figure out what's going on.

Coding Styles-Standards and Guidelines

- **Purpose of Coding Standards:**
  - **Uniform Appearance**: A coding standard gives a uniform appearance to the codes written by different engineers.
  - **Readability and Maintainability**: It improves the readability and maintainability of the code and reduces complexity.
  - **Error Detection**: It helps in code reuse and makes error detection easier.
  - **Promotes Good Practices**: It promotes sound programming practices and increases the efficiency of the programmers.

Coding Styles-Standards and Guidelines

- Some Standards
  - **Limited use of globals**: These rules tell about which types of data that can be declared global and the data that can't be

Coding Styles-Standards and Guidelines

- Some Standards
  - **Standard headers for different modules**: For better understanding and maintenance of the code, the header of different modules should follow some standard format and information. The **header format must contain below things** that is being used in various companies:
  - **Name** of the module
  - **Date** of module creation
  - **Author** of the module
  - **Modification** history
  - **Synopsis** of the module about what the module does
  - **Different functions supported** in the module along with their input output parameters
  - **Global variables accessed** or modified by the module

Coding Styles-Standards and Guidelines

- Some Standards
  - **Naming conventions for local variables, global variables, constants and functions**: Some of the naming conventions are given below:
    - **Meaningful and understandable variables name** helps anyone to understand the reason of using it.
    - **Local variables should be named using camel case** lettering starting with small letter (e.g. localData) whereas **Global variables names should start with a capital letter (e.g. GlobalData)**. **Constant names** should be formed using **capital letters only** (e.g. CONSDATA).
    - It is better **to avoid the use of digits** in variable names.
    - The names of the function should be written in camel case starting with small letters.
    - The name of the function must describe the reason of using the function clearly and briefly.

Coding Styles-Standards and Guidelines

- Some Standards
- **Indentation**:
- Proper **indentation is very important to increase the readability** of the code. For making the code readable, programmers should use White spaces properly. Some of the spacing conventions are given below:
  - There must be a **space after giving a comma** between two function arguments.
  - Each nested **block should be properly indented** and spaced.
  - Proper Indentation should be there at the b**eginning and at the end of each block** in the program.
  - All braces should start from a new line and the code following the end of braces also start from a new line.

Coding Styles-Standards and Guidelines

- Some Standards
- **Error return values and exception handling conventions:**
- All functions that encountering an error condition should **either return a 0 or 1 for simplifying the debugging**.
- On the other hand, Coding guidelines give some general suggestions regarding the coding style that to be followed for the betterment of understandability and readability of the code.

Coding Styles-Standards and Guidelines

- Coding Guidelines:
- **Avoid using a coding style that is too difficult to understand**: Code should be easily understandable. The complex code makes maintenance and debugging difficult and expensive.

- **Avoid using an identifier for multiple purposes**: Each variable should be given a descriptive and meaningful name indicating the reason behind using it. This is not possible if an identifier is used for multiple purposes and thus it can lead to confusion to the reader. Moreover, it leads to more difficulty during future enhancements.

Coding Styles-Standards and Guidelines

- Coding Guidelines:
- **Code should be well documented**: The code should be **properly commented for understanding easily**. Comments regarding the statements increase the understandability of the code.
- **Length of functions should not be very large:** Lengthy functions are very difficult to understand. That's why functions should be small enough to carry out small work and lengthy functions should be broken into small ones for completing small tasks.
- **Try not to use GOTO statement**: GOTO statement makes the program unstructured, thus it reduces the understandability of the program and also debugging becomes difficult.

Coding Styles-Standards and Guidelines

- Advantages of Coding Guidelines:
  - Coding guidelines **increase the efficiency of the software** and reduces the development time.
  - Coding guidelines **help in detecting errors** in the early phases, so it helps to reduce the extra cost incurred by the software project.
  - If coding guidelines are maintained properly, then the software code **increases readability and understandability** thus it reduces the complexity of the code.
  - It reduces the hidden cost for developing the software

Software Documentation

- It is a **written text that accompanies** a software program.
- It's like a **guidebook that makes it easier** for everyone involved in the project.
- It can include **different types of information**, like API documentation, build notes, or help content.
- It's a key part of the process of developing computer code.
- People who work with code often care a lot **about the value, usage, and quality of the documentation** during the development and maintenance of the software.

Software Documentation - Types

- **Requirement Documentation**: It is the description of **how the software shall perform and which environment setup** would be appropriate to have the best out of it. These are **generated while the software is under development and is supplied to the tester** groups too.
- **Architectural Documentation**: Architecture documentation is a special type of documentation that **concerns the design**. It **contains very little code and is more focused on the components of the system, their roles, and working**. It also shows the data flow throughout the system.
- **Technical Documentation**: These **contain the technical aspects of the software like API, algorithms**, etc. It is prepared mostly for software developers.
- **End-user Documentation**: As the name suggests these are made for the end user. It **contains support resources** for the end user.

Software Documentation - Principle

- **Write from Reader's Point of View**: Keep the target audience in mind when writing software documentation. Use simple language and domain-specific terminologies. Organize the content clearly and include a glossary for complex terms.
- **Avoid Unnecessary Repetition**: Use hyperlinking and backlinking to avoid redundancy. Each piece of information should be stored as an individual unit in the back-end database.
- **Avoid Ambiguity**: Ensure all information in the documentation is clear and precise. Define terminologies used in different contexts explicitly to avoid confusion among stakeholders, developers, and maintainers.
- **Follow a Standard Organization**: Maintain professionalism, accuracy, and precision in the document by following certain principles and referencing other software documentations

Software Documentation - Principle

- **Record a Rationale**: Include a rationale section, written by the developer or designer, explaining why certain design or development decisions were made.
- **Keep the Documentation Updated**: Regularly update the documentation to reflect software updates, but only add valuable and relevant content.
- **Review Documentation**: Regularly review the documentation and make necessary changes based on feedback from senior architects to align with its purpose of educating and spreading knowledge

Software Documentation - Advantages

- helps in keeping the track of all aspects of an application and also improves the quality of the software product.
- Helps development teams during development.
- Helps end-users in using the product.
- Improves overall quality of software product
- It cuts down duplicative work.
- Makes easier to understand code.
- Helps in establishing internal coordination in work

Software Documentation - Disadvantages

- The documenting code is time-consuming.
- The software development process often takes place under time pressure, due to which many times the documentation updates don’t match the updated code.
- The documentation has no influence on the performance of an application.
- Documenting is not so fun, it’s sometimes boring to a certain extent.

Modern Programming Language - Features

- **Type Checking**:
  - Type checking is the process of verifying and enforcing the constraints of types.
  - It checks the type of objects and reports a type error in case of a violation.
  - Type checking can be static (performed at compile time) or dynamic (performed at runtime).

Modern Programming Language - Features

- **User-Defined Data Types:**
  - User-defined data types are data types that are defined by the user themselves.
  - These data types are derived from existing data types and allow users to create customized data types that suit their specific needs

Modern Programming Language - Features

- **Data Abstraction:**
  - Data abstraction is a powerful feature in object-oriented programming that **hides the details of how data is implemented,** allowing programmers to focus on the essential features of the data without getting bogged down in the details.
  - It provides a clear separation between the abstract properties of a data type and the concrete details of its implementation.

Modern Programming Language - Features

- **Exception Handling**:
  - Exception handling is a mechanism to handle runtime errors.
  - It allows the program to continue its execution or terminate gracefully when an error occurs.
  - Exception handling techniques can be categorized into programmed exception handling and default exception handling.

Modern Programming Language - Features

- **Concurrency Mechanism:**
  - Concurrency in programming refers to the ability of a system to **execute multiple tasks simultaneously.**
  - It's a key feature in modern programming languages, **enabling efficient utilization of system resources** and improving performance in multi-core and distributed systems.

7 steps of Software Implementation

- **Step 1: Project Kick-off**
  - This is the first step where all stakeholders, including the tech team and end-users, come together to set clear objectives and achievable timelines for the project.

- **Step 2: Data Migration**
  - When switching or upgrading systems, it’s important to transfer all data from the current sources to the new system, ensuring no valuable information is lost and data integrity is maintained.

7 steps of Software Implementation

- **Step 3: Configuration and Customization**
  - Every business is unique, so while the core functionalities of the software might be the same, it’s crucial to adapt it to fit your specific business processes. Balance customization with future software updates to ensure compatibility.
- **Step 4: System Integration**
  - In the interconnected tech world, software rarely works in isolation. It’s critical to ensure that the new software can integrate seamlessly with existing systems to enhance workflow continuity and reduce disruptions.

7 steps of Software Implementation

- **Step 5: User Training**
  - The success of software implementation is not just about the technology, but also about the people using it. Provide **tailored training sessions for different user roles** to speed up the adoption rate. Use hands-on practice and real-life scenarios to make these sessions more effective.

7 steps of Software Implementation

- **Step 6: Testing and Quality Assurance**
  - Before the software goes live, conduct rigorous testing to ensure it meets all requirements and is free from bugs. User acceptance testing (UAT) provides feedback on real-world performance, helping to refine the system before its final deployment.
- **Step 7: Deployment and Go-live**
  - Deploying the software is a significant change. Consider a phased roll-out approach, especially for large organizations. After deployment, provide continuous monitoring and support to address any issues that arise.

Implementation Challenges and its resolution.

- **Misaligned Expectations**: This occurs when the software implementation team and the vendor have different expectations. To avoid this, there must be strong communication between your internal project team and the vendor. **Defining the milestones and deliverables required during the planning stage** can make the implementation stage smoother.
- **Data Integrity**: During software implementation, **data migration from an existing system to a new one is often required. Ensuring data integrity throughout this process** is critical. Understanding the level of interoperability between the two systems can help ensure data security, prevent data loss during migration, and uphold privacy standards.
- **Lack of Workflow Visibility**: Many development teams suffer from a vague understanding of the engineering processes within their organization. This can stem from ineffective communication, lack of effective team collaboration, and broken SDLC processes. Investing in tools that empower your teams to make data-driven decisions can combat this issue.

Implementation Challenges and its resolution.

- **Lack of Standard Organization**: To maintain the professionalism, accuracy, and precision of the document, certain principles must be followed. **Taking reference from other software documentations can aid in organizing and structuring** the content of the documentation in a much productive and organized way.